

98. 验证二叉搜索树

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium	(Not Available)	(Not Available)	-	-	0

- **Tags:** 树, 深度优先搜索, 二叉搜索树, 二叉树
- **Companies:** (Not Available)

给你一个二叉树的根节点 `root` , 判断其是否是一个有效的二叉搜索树。

有效二叉搜索树定义如下:

- 节点的左子树只包含 **小于**当前节点的数。
- 节点的右子树只包含 **大于**当前节点的数。
- 所有左子树和右子树自身必须也是二叉搜索树。

示例 1:

输入 : `root = [2,1,3]`

输出 : `true`

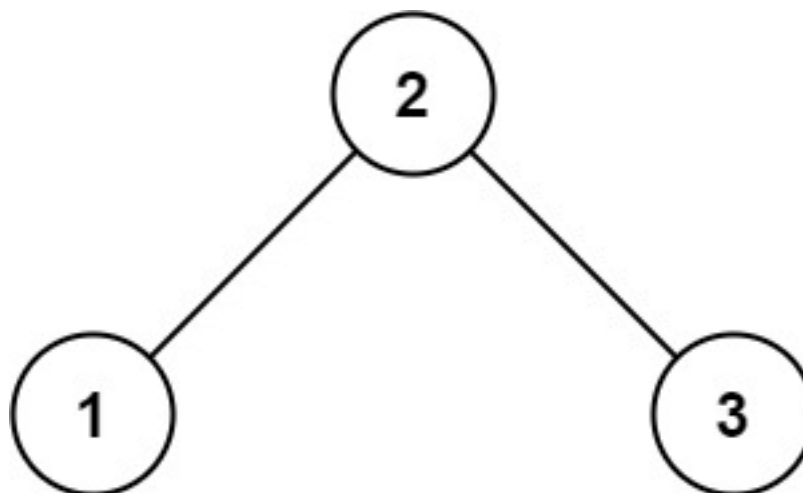


Figure 1: img

示例 2:

输入 : `root = [5,1,4,null,null,3,6]`

输出 : `false`

解释 : 根节点的值是 5 , 但是右子节点的值是 4 。

提示:

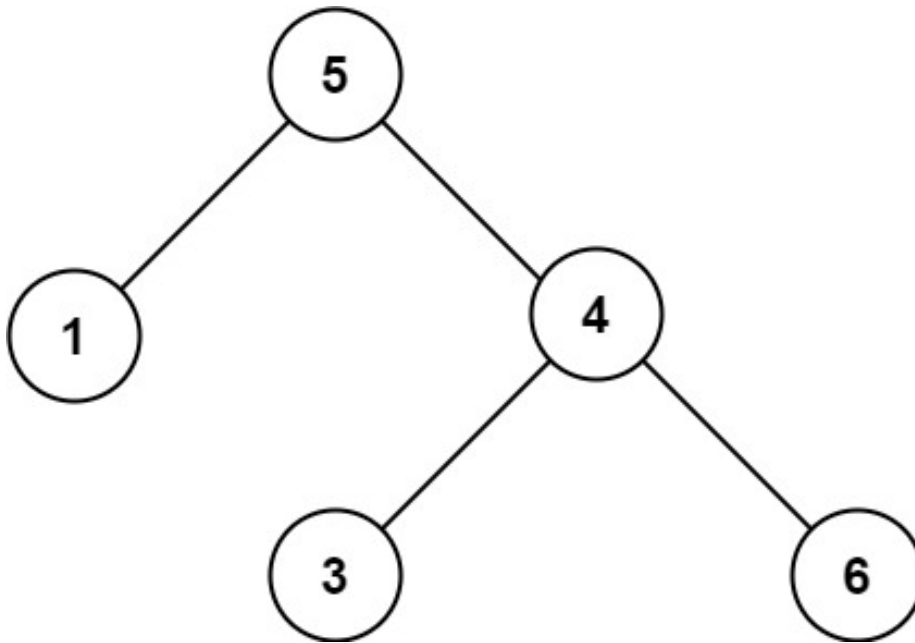


Figure 2: img

- 树中节点数目范围在 $[1, 10^4]$ 内
- $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$

Discussion | Solution

解决方法

1. 问题理解

目标：判断给定的二叉树是否是有效的二叉搜索树 (BST)。**BST 定义：** - 左子树所有节点值 $<$ 根节点值。 - 右子树所有节点值 $>$ 根节点值。 - 左右子树也必须是 BST。

关键陷阱：仅仅比较一个节点与其直接左右子节点的值是不够的。必须保证整个左子树的所有节点都小于该节点，整个右子树的所有节点都大于该节点。例如，对于 $[5, 1, 4, \text{null}, \text{null}, 3, 6]$ ，虽然 $4 > 1$ 且 $4 < 5$ ，但 $3 < 5$ 却出现在了 5 的右子树中，因此不是 BST。

2. 思路分析

方法一：递归 (DFS) - 传递上下界 我们可以使用递归来验证 BST 的性质。在递归过程中，我们需要维护每个节点允许的取值范围（上界和下界）。

1. 定义递归函数 `isValid(node, lower_bound, upper_bound)`:
 - **功能**: 判断以 `node` 为根的子树是否是有效的 BST, 并且其所有节点的值都在 $(lower_bound, upper_bound)$ 这个开区间内。
 - **基本情况 (Base Case)**: 如果 `node` 为 `null`, 空树是有效的 BST, 返回 `true`。
 - **验证当前节点**:
 - 检查 `node.val` 是否满足范围要求: `node.val > lower_bound` 且 `node.val < upper_bound`。如果不满足, 直接返回 `false`。
 - **递归步骤 (Recursive Step)**:
 - 递归验证左子树: 左子树的所有节点必须小于当前节点 `node.val`, 同时也要大于 `lower_bound`。所以调用 `isValid(node.left, lower_bound, node.val)`。
 - 递归验证右子树: 右子树的所有节点必须大于当前节点 `node.val`, 同时也要小于 `upper_bound`。所以调用 `isValid(node.right, node.val, upper_bound)`。
 - 只有当当前节点值有效, 并且左右子树的递归调用都返回 `true` 时, 才返回 `true`。否则返回 `false`。
2. 主函数调用:
 - 调用 `isValid(root, -infinity, +infinity)` 开始验证。初始时, 根节点没有范围限制, 所以下界是负无穷, 上界是正无穷。可以使用 `float('-inf')` 和 `float('inf')` 或足够大/小的整数表示。

方法二: 中序遍历 (DFS) BST 的一个重要性质是: **中序遍历的结果是一个严格递增的序列**。我们可以利用这个性质来验证。

1. **执行中序遍历**: 对给定的二叉树进行中序遍历 (左 -> 根 -> 右)。
2. **存储遍历结果**: 将遍历得到的节点值存储在一个列表中。
3. **检查递增性**: 检查该列表是否是严格递增的。即遍历列表, 比较相邻元素 `list[i]` 和 `list[i-1]`, 如果发现 `list[i] <= list[i-1]`, 则说明不是 BST, 返回 `false`。如果遍历完整个列表都没有发现非递增情况, 则返回 `true`。

优化: 我们不必存储整个中序遍历结果。可以在遍历过程中实时比较。

1. **初始化**: 设置一个变量 `prev_val` 用于记录中序遍历中前一个节点的值, 初始设为负无穷 (`float('-inf')`)。
2. 定义递归函数 `inorder_check(node)`:
 - **功能**: 对以 `node` 为根的子树进行中序遍历检查。
 - **基本情况 (Base Case)**: 如果 `node` 为 `null`, 返回 `true`。
 - **递归左子树**: `left_valid = inorder_check(node.left)`。如果左子树检查失败 (`not left_valid`), 直接返回 `false`。
 - **检查当前节点**: 比较当前节点值 `node.val` 与前一个节点值 `prev_val`。如果 `node.val <= prev_val`, 说明不满足递增性, 返回 `false`。
 - **更新 prev_val**: 将 `prev_val` 更新为当前节点的值 `node.val`。注意: `prev_val` 需要是全局变量或通过引用/闭包传递, 以便在递归调用间保持状态。
 - **递归右子树**: `right_valid = inorder_check(node.right)`。返回 `right_valid` 的结果。
3. 主函数调用:
 - 调用 `inorder_check(root)`。

迭代实现中序遍历检查：也可以使用迭代（栈）的方式实现中序遍历，并在访问节点时进行比较。

1. **初始化：** `stack = []`, `curr = root`, `prev_val = float('-inf')`。
2. **迭代循环：** `while curr or stack:`
 - 一路向左，将节点压栈：`while curr: stack.append(curr); curr = curr.left`。
 - 弹出节点访问：`node = stack.pop()`。
 - 比较：`if node.val <= prev_val: return False`。
 - 更新：`prev_val = node.val`。
 - 转向右子树：`curr = node.right`。
3. **返回结果：**循环正常结束，返回 `true`。

3. 代码实现 (Python)

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

from typing import Optional
import math # 或者 float('-inf'), float('inf')

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    # 方法一：递归 (DFS) - 传递上下界
    def isValidBSTRecursive(self, root: Optional[TreeNode]) -> bool:
        """
        使用递归和上下界判断是否为有效 BST
        """
        def isValid(node, lower_bound, upper_bound):
            if not node:
                return True

            # 检查当前节点值是否在有效范围内
            if not (lower_bound < node.val < upper_bound):
                return False

            # 递归检查左右子树，并更新范围
            # 左子树：上界变为当前节点值
            return isValid(node.left, lower_bound, node.val) and \
                   isValid(node.right, node.val, upper_bound)
```

```

        # 右子树: 下界变为当前节点值
        return isValid(node.left, lower_bound, node.val) and \
            isValid(node.right, node.val, upper_bound)

    # 初始调用, 范围是 (-inf, +inf)
    return isValid(root, float('-inf'), float('inf'))

# 方法二: 中序遍历 (迭代)
def isValidBSTInorder(self, root: Optional[TreeNode]) -> bool:
    """
    使用中序遍历 (迭代) 判断是否为有效 BST
    """
    stack = []
    curr = root
    prev_val = float('-inf') # 记录前一个节点的值

    while curr or stack:
        # 一路向左
        while curr:
            stack.append(curr)
            curr = curr.left

        # 弹出节点访问
        node = stack.pop()

        # 检查是否满足递增性
        if node.val <= prev_val:
            return False

        # 更新前一个节点的值
        prev_val = node.val

        # 转向右子树
        curr = node.right

    return True # 遍历完成且都满足递增性

# 最终提交可以选择其中一种方法
def isValidBST(self, root: Optional[TreeNode]) -> bool:
    # return self.isValidBSTRecursive(root)
    return self.isValidBSTInorder(root)

```

4. 复杂度分析

- 方法一 (递归 - 上下界):

- 时间复杂度: $O(N)$, 其中 N 是树中的节点数。每个节点被访问一次。

- 空间复杂度: $O(H)$, 其中 H 是树的高度。空间消耗主要来自递归调用栈。最坏情况下 (链状树) 为 $O(N)$, 平均情况下 (平衡树) 为 $O(\log N)$ 。
- **方法二 (中序遍历 - 迭代):**
 - 时间复杂度: $O(N)$ 。每个节点被访问 (入栈出栈) 一次。
 - 空间复杂度: $O(H)$ 。栈中最多存储 H 个节点。最坏情况下为 $O(N)$, 平均情况下为 $O(\log N)$ 。

5. 如何在面试中讲解

1. **明确 BST 定义与陷阱:**
 - “首先明确有效 BST 的定义: 左 < 根 < 右, 且左右子树也必须是 BST。”
 - “强调易错点: 不能只比较节点和它的直接孩子, 必须保证整个左子树都小于节点, 整个右子树都大于节点。”
2. **方法一: 递归与上下界:**
 - “一个有效的方法是递归, 并在递归时传递每个节点允许的取值范围 (下界 lower 和上界 upper)。”
 - “定义 `isValid(node, lower, upper)`:
 - Base Case: node 为空, 返回 true。
 - 检查 `node.val` 是否在 $(lower, upper)$ 范围内, 如果不在, 返回 false。
 - 递归调用左子树: `isValid(node.left, lower, node.val)`, 上界更新为当前节点值。
 - 递归调用右子树: `isValid(node.right, node.val, upper)`, 下界更新为当前节点值。
 - 只有当前节点有效且左右递归都返回 true 时, 才返回 true。”
 - “初始调用 `isValid(root, -inf, +inf)`。”
 - “分析复杂度: 时间 $O(N)$, 空间 $O(H)$ 。”
3. **方法二: 中序遍历:**
 - “利用 BST 的关键性质: 中序遍历结果是严格递增的。”
 - “思路一 (简单但不优): 先中序遍历存入数组, 再检查数组是否严格递增。时间 $O(N)$, 空间 $O(N)$ 。”
 - “思路二 (优化): 在迭代中序遍历的过程中实时检查。使用栈进行迭代, 维护一个 `prev_val` 记录上一个访问节点的值。”
 - “迭代步骤: 一路向左入栈 -> 弹出节点 node -> 比较 `node.val` 与 `prev_val` (如果 `node.val <= prev_val` 则返回 false) -> 更新 `prev_val = node.val` -> 处理右子树。”
 - “分析复杂度: 时间 $O(N)$, 空间 $O(H)$ 。”
4. **对比与选择:**
 - “两种方法时间复杂度相同。空间复杂度也类似 (取决于树高)。”
 - “递归方法概念上更直接对应 BST 的定义。中序遍历方法利用了 BST 的一个重要性质, 迭代实现可以避免递归深度问题。”
5. **边界条件:**
 - “处理空树。”
 - “注意节点值可能等于上下界的情况 (根据定义, BST 不允许相等, 所以用开区间)。”
 - “中序遍历方法中 `prev_val` 的初始值设为负无穷。”